

POLITECNICO
MILANO 1863

REPORT RETI LOGICHE

Salvatore Grasso
Codice Persona - 10804762
Matricola - 333739

Anno accademico 2023/2024

Indice

1 Introduzione **2**

2 Architettura **4**
2.1 Registro 4
2.2 Credibility Counter 4
2.3 Word Counter 5
2.4 Flip-Flop T 5
2.5 Sommatore 6
2.6 Encoder 6
2.7 FSM 6
2.8 Schema Circuitale 7
2.9 Sulla FSM e la riduzione degli stati 8

3 Risultati Sperimentali **9**
3.1 Report Utilization 9
3.2 Report timing 9
3.3 Simulazioni 10

4 Conclusioni **11**

A Testbench Generator **12**

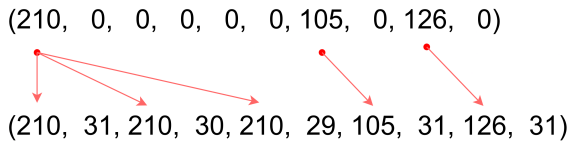
Capitolo 1

Introduzione

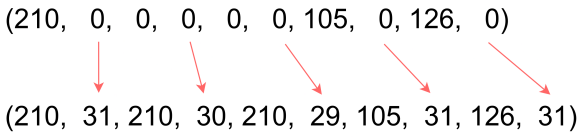
Lo scopo del progetto è sviluppare una rete logica sequenziale in grado di eseguire delle operazioni specificate su una sequenza di K parole W con valori compresi fra 0 e 255. L'insieme delle parole, contenuto in memoria a partire da un indirizzo denominato ADD ogni due byte, può essere interpretato come un gruppo di letture sequenziali provenienti da un sensore. A questo proposito è importante notare come alla codifica 0 non corrisponda un valore numerico, ma l'informazione della non specificità della lettura eseguita.

La sequenza doveva essere interpretata secondo le seguenti specifiche: ad ogni parola doveva essere associato, codificato come un numero da 0 a 31, un indice di credibilità: se la parola assume un valore diverso da zero allora l'indice ad essa associato viene fissato al massimo valore, in caso contrario la parola viene sostituita con l'ultima letta diversa da zero, mentre la credibilità associata viene decrementata di uno fino ad un minimo di zero. Nel caso la sequenza iniziasse con 0, fino ad una parola con un valore non nullo viene associato un indice di credibilità nullo a ogni parola.

La specifica può essere vista come suddivisa in due parti: una propagativa e una generativa. La prima corrisponde alla sovrascrittura dei valori non specificati con l'ultimo valore diverso da zero letto, mentre la seconda contiene le specifiche per la creazione degli indici di affidabilità a partire da una parola della sequenza.



(a) Parte propagativa



(b) Parte generativa

Alcune possibili sequenze sono:

```
( 78,   0, 222,   0,   0,   0,   0,   0,   0,   0)
( 78,  31, 222,  31, 222,  30, 222,  29, 222,  28)

(  0,   0,  50,   0,   0,   0, 210,   0,  53,   0)
(  0,   0,  50,  31,  50,  30, 210,  31,  53,  31)
```

I segnali della rete logica da progettare sono descritti brevemente nella tabella sottostante.

Nome	Tipo	Interazione	Descrizione
i_clk (1 bit)	Input	//	Segnale per la sincronizzazione della rete
i_rst (1 bit)	Input	Utente	Segnale di reset del sistema
i_start (1 bit)	Input	Utente	Quando alto indica la partenza dell'elaborazione
i_add (16 bit)	Input	Utente	Contiene l'indirizzo di memoria dal quale parte la sequenza di dati
i_k (10 bit)	Input	Utente	Codifica in binario senza segno del numero di parole da elaborare
o_done (1 bit)	Output	Utente	Indica la fine dell'elaborazione, rimane alto finché i_start è alto
o_mem_addr (16 bit)	Output	Memoria	L'indirizzo di memoria in cui andare a scrivere o leggere
i_mem_data (8 bit)	Input	Memoria	La parola contenuta nell'indirizzo specificato in o_mem_addr
o_mem_data (8 bit)	Output	Memoria	La parola da andare a scrivere in o_mem_addr
o_mem_en (1 bit)	Output	Memoria	Alto quando si vuole interagire con la memoria, basso altrimenti
o_mem_we (1 bit)	Output	Memoria	Alto se si vuole scrivere in memoria, basso altrimenti.

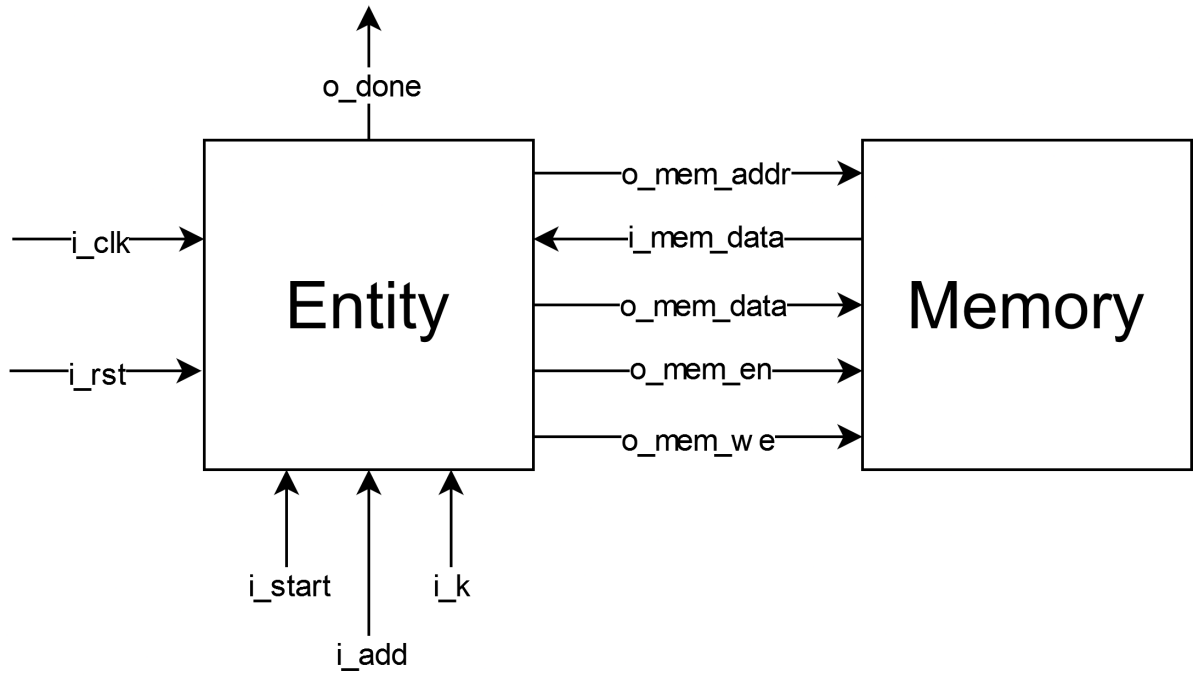


Figura 1.2: Segnali di connessione

Capitolo 2

Architettura

Per risolvere il problema, la rete fa uso di vari componenti, tra i quali quelli centrali al funzionamento sono: un registro, un contatore in modulo 32 che è usato per associare la credibilità di una parola e un contatore che tenga conto di quante parole siano già state elaborate. In concordanza con la specifica che ogni segnale tranne il reset deve essere interpretato sul fronte di salita del clock alcuni dei componenti sono dotati di un segnale di reset sincrono per ripartire con una nuova sequenza dopo la prima.

2.1 Registro

Il registro progettato è sincrono e opera con un segnale di attivazione. Al momento del reset, il componente produce in uscita un segnale di soli zeri, successivamente, in corrispondenza del fronte di salita del clock, se il segnale di attivazione è alto in uscita è trasmesso il segnale di input, ma solo se questo è diverso da tutti zero. Questo consente alla rete di memorizzare una parola per la parte propagativa della specifica. In aggiunta, il componente è dotato di un segnale di restart sincrono, che pone l'uscita a tutti zero per le successive elaborazioni dopo il reset

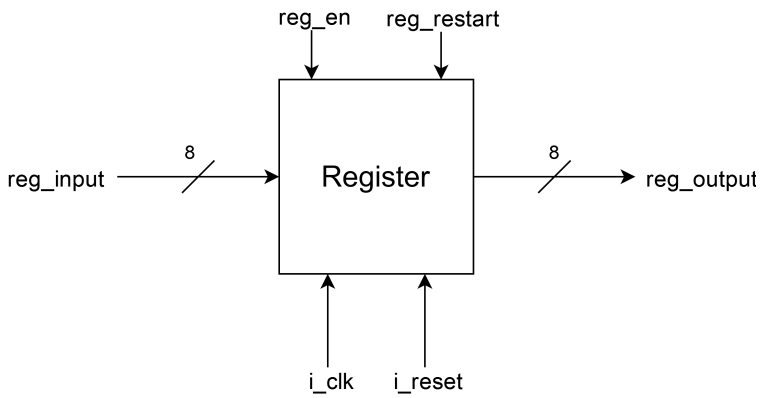


Figura 2.1: Registro

2.2 Credibility Counter

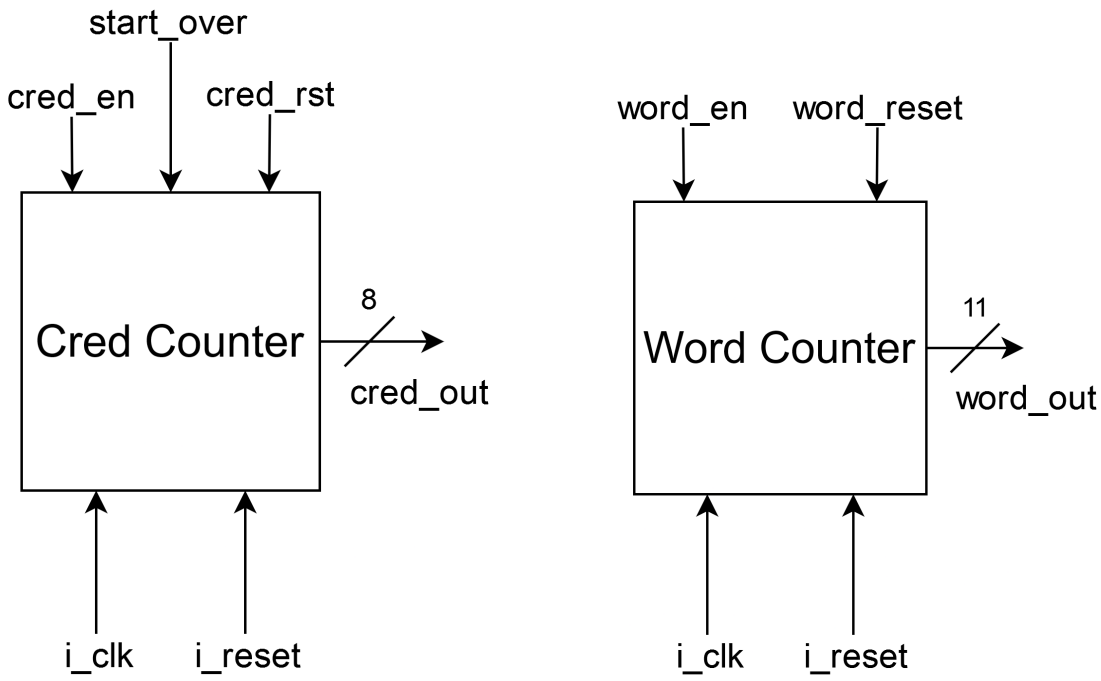
Si tratta di un contatore decrescente sincrono in modulo 32 dotato di segnale di attivazione. Usato per calcolare il valore di credibilità associato ad ogni parola della sequenza.

Il componente è dotato di un segnale denominato "start_over" che, in corrispondenza del fronte di salita del clock e di un segnale di attivazione alto, pone l'uscita alla codifica in binario naturale di 31. Una volta che il contatore raggiunge lo zero, la sua uscita rimane a quel valore fino all'arrivo di un segnale di "start_over" o reset. È inoltre presente un segnale di reset sincrono nominato "cred_restart"

2.3 Word Counter

Un semplice contatore crescente sincrono dotato di un segnale di attivazione. Utilizzato per calcolare gli indirizzi di memoria delle parole della sequenza e per alzare il segnale di "o_done" quando l'elaborazione è completata.

Sapendo che il numero massimo di parole da elaborare è $2^{10} - 1$, il massimo numero di indirizzi in memoria occupati da una sequenza è $2^{11} - 2$, per questo motivo il contatore deve essere in modulo 2048. Anche questo componente è dotato di un segnale di reset sincrono



(a) Credibility Counter

(b) Word Counter

Figura 2.2: Contatori

2.4 Flip-Flop T

Componente usato per distinguere in quale fase della specifica si trovi la rete ad un determinato istante.

Al momento del reset t_out è basso. Il segnale di toggle t_in diventa alto solo quando si è finito con la fase di elaborazione corrente. Di conseguenza, se il segnale t_out è basso ci troviamo nella fase propagativa della specifica, se è alto nella fase generativa

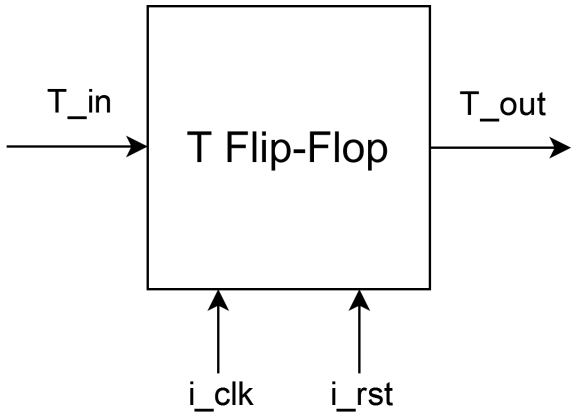


Figura 2.3: Flip-Flop T

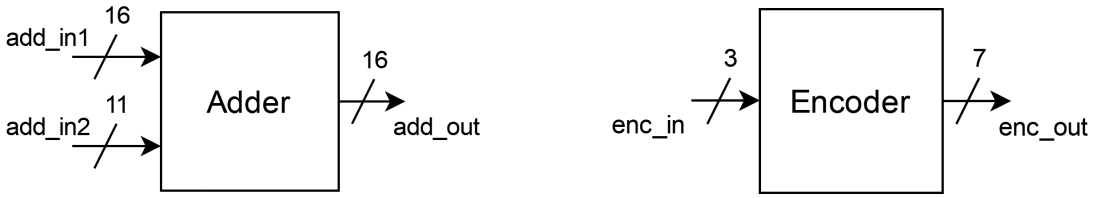
2.5 Sommatore

Un normale sommatore combinatorio in binario naturale. Permette di sapere con quale cella di memoria interagire sommando i segnali word_out e i_add.

2.6 Encoder

Semplice encoder combinatorio. Ha il compito di assegnare i corretti segnali interni per ogni stadio della FSM.

Per dettagli maggiori sulle uscite dell'encoder si rimanda alla tabella 2.1

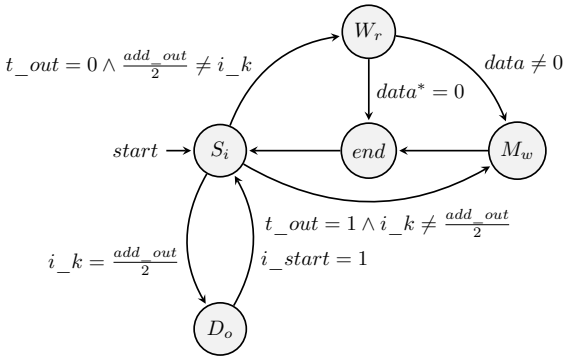


(a) Sommatore (b) Encoder

Figura 2.4: Sommatore e Encoder

2.7 FSM

Si tratta di una macchina a stati finiti di Measly con cinque stati. È il componente centrale della rete ed ha il compito coordinare le varie operazioni eseguite dalla rete attraverso gli stadi



(a) FSM della rete

	t_out		$data$				
	0	1	0	$\neq 0$			
	S_i		W_r		M_w	end	D_o
$done\backslash restart$	0		0		0	0	1
mem_we	0		-		1	-	-
mem_en	1	0	0		1	0	0
reg_en	0		1		0	0	0
$word_en\backslash t_in$	0		0		0	1	0
$cred_en$	0		1		0	0	0
$start_over$	-		0	1	-	-	-

(b) Valore segnali per ogni stato FSM

Figura 2.6: FSM

1. S_i: stato di reset della FSM. Quando $t_out = 0$ si legge la memoria mettendo $mem_en = 1$, se $t_out = 1$ passa allo stadio M_w. Quando $\frac{add_{out}}{2}$** $= i_k$ le parole elaborate sono uguali a quelle richieste quindi si passa a D_o
2. W_r: stato di attesa dopo la lettura per avere la parola dalla memoria, si abilita il registro e se $i_mem_data = 0$ si alza il segnale di $start_over$ e si passa a M_w per la propagazione.
3. M_w: stato di scrittura, si alzano mem_en e mem_we.
4. end: stato di fine elaborazione, si alza $word_en$ cosi da passare all'elaborazione successiva
5. D_o: stato che indica che si è finito di elaborare la sequenza, il segnale di o_done viene alzato e i componenti vengono resettati in modo sincrono

*$data = i_{mem_data}$

**operazione fatta shiftando di un bit a destra nella rete

L'output della Finite State Machine (FSM) è collegato all'input dell'encoder, che si occupa di inviare i segnali necessari per l'elaborazione successiva. Dal momento che ci sono un totale di 7 configurazioni di segnali distinti $[2(S_i)+2(W_r)+1(M_w)+1(end)+1(D_o)]$, è necessario che l'input dell'encoder sia composto da almeno $\lceil \log_2(7) \rceil = 3$ bit

			done	mem_{we}	mem_{en}	reg_{en}	$word_{en}$	$cred_{en}$	$start_over$	
enc_{in}			enc_{out}							
0	0	0	0	0	1	0	0	0	-	S_{i_1}
0	0	1	0	-	0	1	0	1	0	W_{m_1}
0	1	0	0	-	0	1	0	1	1	W_{m_2}
0	1	1	0	1	1	0	0	0	-	M_w
1	0	0	0	-	0	0	1	0	-	end
1	0	1	1	-	0	0	0	0	-	D_o
1	1	0	0	-	0	0	0	0	-	S_{i_2}
1	1	1	X	X	X	X	X	X	X	

Tabella 2.1: tabella verità encoder

2.8 Schema Circuitale

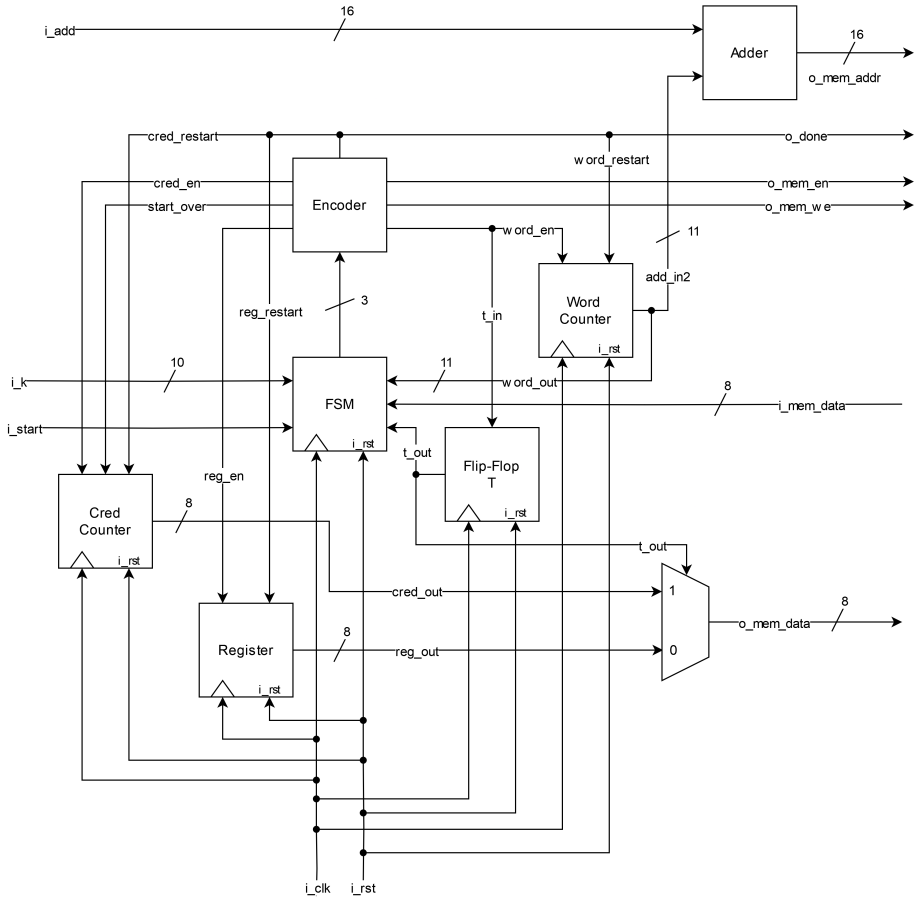


Figura 2.7: schema circuitale

2.9 Sulla FSM e la riduzione degli stati

Durante le fasi iniziali di progettazione si era risolto il problema con una FSM a otto stadi. Tuttavia, dopo ulteriori riflessioni, si è notato che il numero degli stadi della FSM poteva essere ridotto ulteriormente ai cinque della macchina a stati finiti corrente.

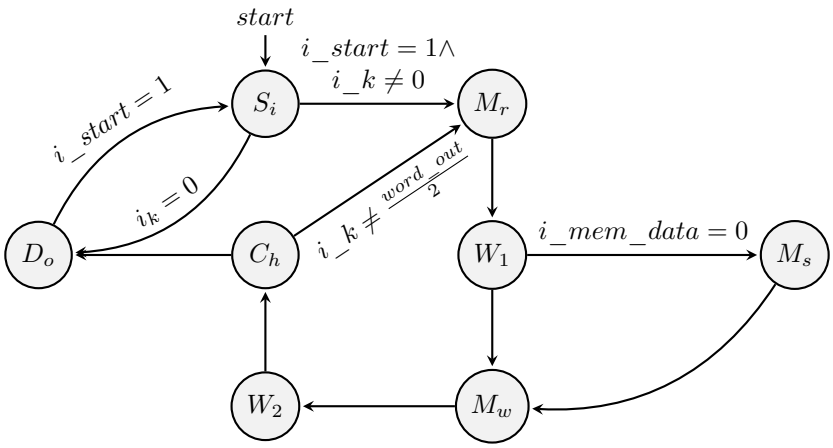


Figura 2.8: Macchina a stati finiti originale

1. S_i: stato di reset della macchina
2. M_r: stato di lettura, viene posto il segnale o_mem_en a 1
3. W_1: stato di attesa, viene alzato il segnale $word_en$ e il segnale reg_en viene posto a 1 per memorizzare il dato della memoria in caso non sia uno zero
4. M_s: stato di sovrascrittura della memoria, se il dato in memoria era zero viene sovrascitto con l'ultimo dato letto diverso da zero
5. M_w: stato di scrittura della memoria, viene scritto in memoria l'output di $cred_counter$
6. W_2: secondo stato di attesa, viene posto a uno il segnale di $word_en$ per aumentare di uno le parole elaborate
7. C_h: stato di controllo, se $i_k = \frac{add_out}{2}$ allora si passa D_o altrimenti si passa in M_r
8. D_o: stato finale, viene alzato il segnale o_done

Nella macchina attuale si è eliminato il secondo stato di attesa e lo stato M_s. Inoltre, lo stato di lettura e reset del sistema sono stati uniti. Questo è stato possibile grazie all'introduzione del flip-flop che permette di far capire alla FSM se bisogna propagare la parola letta o generare il valore di credibilità.

Anche la FSM attuale può essere ridotta, si potrebbe, infatti, eliminare lo stato di end, ma questo vorrebbe dire scrivere in memoria ogni parola letta comportando un dispendio di energia maggiore. Si è quindi deciso di non andare avanti nell'ulteriore riduzione degli stadi e, in questo modo, di avere un compromesso tra efficienza energetica e prestazioni della rete.

Capitolo 3

Risultati Sperimentali

La rete realizzata è stata sintetizzata in Vivado 2022.2 su una FPGA della famiglia Artix 7 (xc7a200tfbg484-1). Successivamente è stato codificato un generatore di testbench in Python per testare la rete, prestando particolare attenzione a eventuali casi limite. Alla fine, attraverso l'uso dei comandi *report_utilization* e *report_timing* si sono controllati l'utilizzo della FPGA e i ritardi dell'entità progettata e ci si è assicurati che questi rispettassero le specifiche.

3.1 Report Utilization

Questo comando è stato usato per verificare l'occupazione delle risorse della rete sulla FPGA su cui è stata fatta la sintesi.

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	64	0	0	134600	0.05
LUT as Logic	64	0	0	134600	0.05
LUT as Memory	0	0	0	46200	0.00
Slice Registers	31	0	0	269200	0.01
Register as Flip Flop	31	0	0	269200	0.01
Register as Latch	0	0	0	269200	0.00
F7 Muxes	0	0	0	67300	0.00
F8 Muxes	0	0	0	33650	0.00

Figura 3.1: Report Utilization

Come si può vedere nella figura 3.1, la rete fa uso di 31 Flip-Flop e 0 Latch, in concordanza con la specifica sulla lettura dei segnali. Dalla tabella si vede anche che sono state utilizzate 64 LUT (Look-Up Table) per eseguire operazione logiche e 0 per memorizzare dati. Si può anche vedere che sono state utilizzate lo 0.05% delle LUT disponibili nella FPGA e lo 0.01% degli Slice Register.

3.2 Report timing

Si è usato questo comando per verificare se la rete rispettasse il vincolo di velocità dei segnali imposto dal clock. A differenza della simulazione comportamentale, infatti, la simulazione sulla sintesi tiene conto anche dei ritardi che si avrebbero utilizzando la FPGA. Le specifiche di progetto richiedevano un segnale di clock con durata di 20ns e duty_cicle del 50%.

```
Max Delay Paths
--------------------------------------------------------------------------------
Slack (MET) :             15.291ns  (required time - arrival time)
  Source:                 word_counter/num_reg[6]/C
  Destination:            credibility_counter/num_reg[0]/CE
  Data Path Delay:        4.327ns

                         required time                          22.041
                         arrival time                           -6.751
  ------------------------------------------------------------------
                         slack                                  15.291
```

Figura 3.2: Report Timing

Il comando fornisce dettagli sullo slack time, ovvero la differenza tra il tempo dettato dal vincolo imposto e il tempo di arrivo del segnale con il ritardo peggiore. Nel caso riportato in figura 3.2 lo slack è positivo, questo vuol dire che il ritardo peggiore della rete è minore del tempo dettato dal vincolo imposto, di conseguenza il comportamento della rete progettata è quello atteso. Sono mostrate anche altre informazioni sul segnale con il massimo ritardo, tra le quali: il punto di partenza del segnale (in questo caso il word counter), il punto di arrivo (il credibility counter) e il tempo per arrivare da un punto all'altro.

3.3 Simulazioni

Per simulare la rete progettata si è costruito un generatore di testbench in Phyton. Questo generatore permette di creare automaticamente testbench che includono vari casi limite, tra i quali:

- caso con il massimo numero di parole ($i_k = 1111111111$)
- caso con il minimo numero di parole ($i_k = 0000000000$)
- caso con parole tutte uguali a zero
- caso con parole tutte diverse da zero
- caso con parole con credibilità che arriva a zero
- caso con una sequenza di parole che inizia con degli zero

Oltre ai casi citati il generatore è anche in grado di testare più sequenze in uno stesso testbench (per più dettagli sul generatore si riporta il codice in appendice A).

Oltre ai casi citati precedentemente, ne sono stati testati anche diversi alzando il segnale i_rst durante l'elaborazione di una sequenza.

Ogni caso è stato eseguito con successo sia nella simulazione comportamentale che nella simulazione post-sintesi funzionale. Automatizzando il processo di testing sono state eseguite con successo più di 10.000 sequenze, aumentando cosi la probabilità di validare il corretto funzionamnento dell'entità sviluppata.

Capitolo 4

Conclusioni

Concludendo, la suddivisione del progetto in due parti distinte, generativa e propagativa, ha consentito la creazione di entità specifiche per la risoluzione del problema. Tuttavia, alcune scelte di design, come il numero di stadi per la FSM non ottimizzato a tre, potrebbero richiedere una revisione futura. È opportuno valutare attentamente queste scelte alla luce dei dati relativi al consumo energetico, specialmente considerando le scritture in memoria aggiuntive.

I risultati sperimentali sull'utilizzo delle risorse della FPGA e sul report timing soddisfano le specifiche del progetto. Inoltre, la copertura dei casi di test, includendo un numero significativo di scenari, rende più ragionevole l'ipotesi che il circuito progettato funzioni in modo corretto.

Appendice A

Testbench Generator

```
import random
from bisect import bisect_left

def binary_search(sequence,number):
  i = bisect_left(sequence,number)
  if(i<len(sequence) and sequence[i]==number):
    return 1
  else:
    return 0

def generate_sequence(n):
  random.seed(n)
  x = random.randint(0,1023)
  length = x
  sequence = []
  for i in range(length):
    if(i%2==0):
      if(random.random()<0.65):
        sequence.append(0)
      else:
        sequence.append(random.randint(1,255))
    else:
      sequence.append(0)
  return sequence

def generate_solution(sequence):
  counter = 0
  backup = 0
  solution = []
  for i in range(len(sequence)):
    if(i%2==0):
      if(sequence[i]==0 and counter>0):
        counter -= 1
      elif(sequence[i]!=0):
        counter = 31
        backup = sequence[i]
      solution.append(backup)
    else:
      solution.append(counter)
  return solution

def toString(sequence):
  scenario = "("
  scenario = scenario + ", ".join("{:3d}".format(num) for num in sequence)
  scenario = scenario + ")"
  return scenario

```

```
48  def gen_tb(tb_name,number):
49    seq_zero = []
50    body = ""
51    body += "library ieee;\nuse ieee.std_logic_1164.all;\nuse ieee.numeric_std.all;\nuse
      ↪  std.textio.all;\n\n"
52    body += ("entity " + tb_name + " is\n" + "end " + tb_name + ";\n\n")
53    body += ("architecture " + tb_name +"_arch "+ " of " + tb_name + " is\n")
54    body += ("\tconstant CLOCK_PERIOD : time := 20 ns;\n\tsignal tb_clk : std_logic :=
      ↪  '0';\n\tsignal tb_rst, tb_start, tb_done : std_logic;\n\tsignal tb_add :
      ↪  std_logic_vector(15 downto 0);\n\tsignal tb_k : std_logic_vector(9 downto 0);\n\n")
55    body += ("\tsignal tb_o_mem_addr, exc_o_mem_addr, init_o_mem_addr : std_logic_vector(15
      ↪  downto 0);\n\tsignal tb_o_mem_data, exc_o_mem_data, init_o_mem_data : std_logic_vector(7
      ↪  downto 0);\n\tsignal tb_i_mem_data : std_logic_vector(7 downto 0);\n\tsignal tb_o_mem_we,
      ↪  tb_o_mem_en, exc_o_mem_we, exc_o_mem_en, init_o_mem_we, init_o_mem_en : std_logic;\n\n")
56    body += ("\ttype ram_type is array (300 downto 0) of std_logic_vector(7 downto 0);\n\tsignal
      ↪  RAM : ram_type := (OTHERS => \"00000000\");\n")
57
58    # SCRITTURA SCENARI
59    for i in range(1,number+1):
60      sequence = generate_sequence(None)
61      solution = generate_solution(sequence)
62      if(len(sequence)==0):
63        body += ("\n\t--SCENARIO NUMERO: "+str(i)+" VUOTO - MEMORIA NON CARICATA\n")
64        seq_zero.append(i)
65      else:
66        body +=  ("\n\tconstant SCENARIO"+str(i)+"_LENGTH : integer := " +
          ↪  str(int(len(sequence)/2)) + ";\n")
67        body += ("\ttype scenario"+str(i)+"_type is array(0 to " + str(len(sequence)-1) + ") of
          ↪  integer;\n")
68        body += ("\tsignal scenario"+str(i)+"_input : scenario"+str(i)+"_type := " +
          ↪  toString(sequence) + ";\n")
69        body += ("\tsignal scenario"+str(i)+"_full : scenario"+str(i)+"_type  := " +
          ↪  toString(solution) + ";\n")
70        body += ("\tconstant SCENARIO"+ str(i)+"_ADDRESS: integer := " +
          ↪  str(random.randint(0,0))+ ";\n")
71
72    #CONFIGURAZIONE DELLA MEMORIA E MEMORY SWAP
73    body += ("\n\tsignal memory_control: std_logic := '0';\n\tshared variable mem_modified:
      ↪  integer := 0;\n")
74    body += ("\tcomponent project_reti_logiche is\n\t\tport (\n\t\t\ti_clk : in
      ↪  std_logic;\n\t\t\ti_rst : in std_logic;\n\t\t\ti_start : in std_logic;\n\t\t\ti_add : in
      ↪  std_logic_vector(15 downto 0);\n\t\t\ti_k   : in std_logic_vector(9 downto
      ↪  0);\n\n\t\to_done : out std_logic;\n\n\t\to_mem_addr : out std_logic_vector(15 downto
      ↪  0);\n\t\ti_mem_data : in  std_logic_vector(7 downto 0);\n\t\to_mem_data : out
      ↪  std_logic_vector(7 downto 0);\n\t\to_mem_we   : out std_logic;\n\t\to_mem_en   : out
      ↪  std_logic\n\t);\n\tend component project_reti_logiche;")
75    body += ("\nbegin\n\tUUT : project_reti_logiche\n\tport map(\n\t\t\ti_clk   =>
      ↪  tb_clk,\n\t\t\ti_rst   => tb_rst,\n\t\t\ti_start => tb_start,\n\t\t\ti_add   =>
      ↪  tb_add,\n\t\t\ti_k     => tb_k,\n\t\t\t\n\t\to_done =>
      ↪  tb_done,\n\t\t\t\n\t\to_mem_addr => exc_o_mem_addr,\n\t\t\ti_mem_data =>
      ↪  tb_i_mem_data,\n\t\t\to_mem_data => exc_o_mem_data,\n\t\t\to_mem_we   =>
      ↪  exc_o_mem_we,\n\t\t\to_mem_en   => exc_o_mem_en\n\t);\n\n\ttb_clk <= not tb_clk after
      ↪  CLOCK_PERIOD/2;")
76    body += ("""\n\n    MEM : process (tb_clk)\n    begin\n        if tb_clk'event and tb_clk =
      ↪  '1' then\n            if tb_o_mem_en = '1' then\n                if tb_o_mem_we = '1'
      ↪  then\n\t               mem_modified := 1;\n
      ↪  RAM(to_integer(unsigned(tb_o_mem_addr))) <= tb_o_mem_data after 1 ns;\n
      ↪  tb_i_mem_data <= tb_o_mem_data after 1 ns;\n                else\n
      ↪  tb_i_mem_data <= RAM(to_integer(unsigned(tb_o_mem_addr))) after 1 ns;\n
      ↪  end if;\n            end if;\n        end if;\n    end process;\n""")
```

```
77    body += ("""\n    memory_signal_swapper : process(memory_control, init_o_mem_addr,
      ↪  init_o_mem_data,\n                                        init_o_mem_en,  init_o_mem_we,
      ↪  exc_o_mem_addr,\n                                       exc_o_mem_data, exc_o_mem_en,
      ↪  exc_o_mem_we)\n    begin\n        tb_o_mem_addr <= init_o_mem_addr;\n
      ↪  tb_o_mem_data <= init_o_mem_data;\n        tb_o_mem_en   <= init_o_mem_en;\n
      ↪  tb_o_mem_we   <= init_o_mem_we;\n\n        if memory_control = '1' then\n
      ↪  tb_o_mem_addr <= exc_o_mem_addr;\n           tb_o_mem_data <= exc_o_mem_data;\n
      ↪  tb_o_mem_en   <= exc_o_mem_en;\n            tb_o_mem_we   <= exc_o_mem_we;\n        end
      ↪  if;\n    end process;\n""")
78
79    #CONFIGURAZIONE SCENARI
80    body += ("""\n    create_scenario : process\n    begin\n        wait for 50 ns;\n\n
      ↪  tb_start <= '0';\n        tb_add <= (others=>'0');\n        tb_k   <= (others=>'0');\n
      ↪  tb_rst <= '1';\n        \n        wait for 50 ns;\n        \n        tb_rst <=
      ↪  '0';\n\n""")
81    for i in range(1,number+1):
82      body += ("--CONFIGURAZIONE MEMORIA PER SCENARIO: " + str(i) + "\n")
83
84      if(binary_search(seq_zero,i)==0):
85        body += ("\t\tmemory_control <= '0';\n")
86        body += ("\t\twait until falling_edge(tb_clk);\n")
87        body += ("""        for i in 0 to SCENARIO"""+str(i)+"""_LENGTH*2-1 loop\n
          ↪  init_o_mem_addr<= std_logic_vector(to_unsigned(SCENARIO"""+str(i)+"""_ADDRESS+i,
          ↪  16));\n            init_o_mem_data<=
          ↪  std_logic_vector(to_unsigned(scenario"""+str(i)+"""_input(i),8));\n
          ↪  init_o_mem_en  <= '1';\n            init_o_mem_we  <= '1';\n            wait until
          ↪  rising_edge(tb_clk);   \n        end loop;\n""")
88        body += ("""        wait until falling_edge(tb_clk);\n        memory_control <= '1';\n
          ↪  tb_add <= std_logic_vector(to_unsigned(SCENARIO"""+str(i)+"""_ADDRESS, 16));\n
          ↪  tb_k   <= std_logic_vector(to_unsigned(SCENARIO"""+str(i)+"""_LENGTH, 10));\n
          ↪  tb_start <= '1';\n        while tb_done /= '1' loop                \n            wait
          ↪  until rising_edge(tb_clk);\n        end loop;\n        wait for 5 ns;\n
          ↪  tb_start <= '0';\n\n\n""")
89      else:
90        body += ("""        mem_modified := 0;   \n        wait until rising_edge(tb_clk);\n
          ↪  wait until rising_edge(tb_clk);\n        memory_control <= '1';\n        tb_add <=
          ↪  (others => '0');\n        tb_k <= (others => '0');\n        tb_start <= '1';\n
          ↪  wait for 5ns;\n        while tb_done /= '1' loop\n            wait until
          ↪  rising_edge(tb_clk);\n        end loop;\n        wait for 5ns;\n        tb_start <=
          ↪  '0';\n\n\n""")
91    body += ("\t\t wait;\n\t end process;\n")
92
93    #ASSERT E VERIFICA
94    body += ("""\n    test_routine : process\n    begin\n\n        wait until tb_rst = '1';\n
      ↪  wait for 25 ns;\n        assert tb_done = '0' report "TEST FALLITO o_done !=0 during
      ↪  reset" severity failure;\n        wait until tb_rst = '0';\n\n        wait until
      ↪  falling_edge(tb_clk);\n        assert tb_done = '0' report "TEST FALLITO o_done !=0 after
      ↪  reset before start" severity failure;\n""")
95    for i in range(1,number+1):
96      if(binary_search(seq_zero,i)==0):
```

```
97          body += ("""\n-- SCENARIO NUMERO: """+str(i)+""" \n        wait until
            ↪  rising_edge(tb_start);\n        \n        while tb_done /= '1' loop                \n
            ↪  wait until rising_edge(tb_clk);\n        end loop;\n\n        assert tb_o_mem_en =
            ↪  '0' or tb_o_mem_we = '0' report "TEST FALLITO o_mem_en !=0 memory should not be
            ↪  written after done." severity failure;\n\n        for i in 0 to
            ↪  SCENARIO"""+str(i)+"""_LENGTH*2-1 loop\n            assert
            ↪  RAM(SCENARIO"""+str(i)+"""_ADDRESS+i) =
            ↪  std_logic_vector(to_unsigned(scenario"""+str(i)+"""_full(i),8)) report "TEST FALLITO
            ↪  @ OFFSET=" & integer'image(i) & " expected= " &
            ↪  integer'image(scenario"""+str(i)+"""_full(i)) & " actual=" &
            ↪  integer'image(to_integer(unsigned(RAM(SCENARIO"""+str(i)+"""_ADDRESS + i)))) severity
            ↪  failure;\n        end loop;\n\n        wait until falling_edge(tb_start);\n
            ↪  assert tb_done = '1' report "TEST FALLITO o_done !=0 after reset before start"
            ↪  severity failure;\n        wait until falling_edge(tb_done);\n""")
98        else:
99          body += ("""\n-- SCENARIO NUMERO: """+str(i)+""" \n         wait until
            ↪  rising_edge(tb_start);\n        \n        while tb_done /= '1' loop                \n
            ↪  wait until rising_edge(tb_clk);\n            assert mem_modified=0 report "TEST
            ↪  FALLITO - MEM SCRITTA QUANDO LUNGHEZZA ZERO" severity failure;\n        end loop;\n
            ↪  \n        assert tb_o_mem_en = '0' or tb_o_mem_we = '0' report "TEST FALLITO o_mem_en
            ↪  !=0 memory should not be written after done." severity failure;\n        \n
            ↪  wait until falling_edge(tb_start);\n        assert tb_done = '1' report "TEST FALLITO
            ↪  o_done !=0 after reset before start" severity failure;\n        wait until
            ↪  falling_edge(tb_done);\n""")
100     body += ("""\n\t\tassert false report "Simulation Ended! TEST PASSATO (EXAMPLE)" severity
        ↪  failure;\n    end process;\nend """+tb_name+"""_arch;\n""")
101     with open(tb_name+".vhd","w") as f:
102       f.write(body)
103
104   # Per cambiare lunghezza e/o probabilità nell'ottenere 0 nella sequenza cambiare la
      ↪  generate_sequence
105   # gen_tb("prova.txt",numero sequenze nel testbench)
106   gen_tb("test100",100)
```